AER1415: Computational Optimization
Assignment 3

Zouhair Adam Hamaimou
Student Number: 1004891986

Question 1

Consider the following optimization problem with a single equality constraint:

$$\begin{aligned} \underset{x_1\in\mathbb{R},\ x_2\in\mathbb{R}}{\textbf{Minimize:}} \quad & f(x_1, x_2) = x_1^3 + x_2^3 \\ \textbf{Subject to:} \quad & h(x_1, x_2) = x_1 + x_2 - 1 = 0 \end{aligned}$$

First-order KKT Conditions:

The Lagrangian function for this optimization problem can be written as:

$$\begin{aligned} \mathcal{L}(x_1, x_2, \lambda) &= f(x_1, x_2) - \lambda h(x_1, x_2) \\ &= x_1^3 + x_2^3 - \lambda(x_1 + x_2 - 1) \end{aligned}$$

The first-order KKT conditions are:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x_1} &= 3x_1^2 - \lambda = 0 \\ \frac{\partial \mathcal{L}}{\partial x_2} &= 3x_2^2 - \lambda = 0 \\ \frac{\partial \mathcal{L}}{\partial \lambda} &= x_1 + x_2 - 1 = 0 \end{aligned}$$

Since there is only 1 equality constraint $h(x_1, x_2) = x_1 + x_2 - 1 = 0$, the constraint gradient $\nabla h = \begin{bmatrix} 1 & 1 \end{bmatrix}^T$ forms a one-vector set, and any non-zero vector is linearly independent. So, the regularity condition holds trivially in this case.

Local Minima:

From the first two KKT conditions:

$$3x_1^2 = \lambda = 3x_2^2 \Rightarrow x_1^2 = x_2^2 \Rightarrow x_1 = \pm x_2$$

- Case 1: $x_1 = x_2 \Rightarrow x_1 + x_1 = 1 \Rightarrow x_1 = \frac{1}{2}, x_2 = \frac{1}{2}$
- Case 2: $x_1 = -x_2 \Rightarrow x_1 - x_1 = 1 \Rightarrow 0 = 1$ (Contradiction)

Therefore, the only solution satisfying the KKT conditions is:

$$x_1 = \frac{1}{2}, \quad x_2 = \frac{1}{2}, \quad \lambda = \frac{3}{4}$$

To verify this is a local minimum, we check the second-order conditions. The Hessian of the Lagrangian is:

$$\nabla^2_{xx}\mathcal{L} = \begin{bmatrix} 6x_1 & 0 \\ 0 & 6x_2 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} \text{ at } x_1 = x_2 = \frac{1}{2}$$

The KKT point is indeed a local minimum because the Hessian is positive definite in the null space of ∇h:

$$\text{null}(\nabla h) = \left\{ \mathbf{d} \in \mathbb{R}^2 : \mathbf{d}^T \nabla h = d_1 + d_2 = 0 \right\} = \text{span}\left\{ \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right\}$$

Let $\mathbf{d} = \alpha \begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Then:

$$\mathbf{d}^T \nabla^2_{xx}\mathcal{L}\, \mathbf{d} = \begin{bmatrix} \alpha & -\alpha \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} \alpha \\ -\alpha \end{bmatrix} = 3(\alpha^2 + \alpha^2) = 6\alpha^2 > 0 \quad \text{for all } \alpha \neq 0$$

Hence, all KKT conditions are satisfied and the second-order condition confirms that $\left(\frac{1}{2}, \frac{1}{2}\right)$ is a strict local minimum.

Question 2

Consider the following quadratic programming problem with two equality constraints:

$$\begin{aligned} &\underset{x_1 \in \mathbb{R},\ x_2 \in \mathbb{R},\ x_3 \in \mathbb{R}}{\textbf{Minimize:}} && f(\mathbf{x}) = 3x_1^2 + 2.5x_2^2 + 2x_3^2 + 2x_1x_2 + x_1x_3 + 2x_2x_3 - x_1 - x_2 - x_3 \\ &\textbf{Subject to:} && x_1 + x_3 = 3 \\ &&& x_2 + x_3 = 0 \end{aligned}$$

First-order Optimality Conditions

We write the objective function in the standard quadratic form:

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{Q}\mathbf{x} + \mathbf{c}^T\mathbf{x}$$

Where:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad \mathbf{Q} = \begin{bmatrix} 6 & 2 & 1 \\ 2 & 5 & 2 \\ 1 & 2 & 4 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix}$$

The equality constraints can be written in matrix form as:

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \quad \mathbf{A} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$

The Lagrangian is:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \frac{1}{2}\mathbf{x}^T\mathbf{Q}\mathbf{x} + \mathbf{c}^T\mathbf{x} - \boldsymbol{\lambda}^T(\mathbf{A}\mathbf{x} - \mathbf{b})$$

The first-order optimality conditions are:

$$\nabla_{\mathbf{x}}\mathcal{L} = \mathbf{Q}\mathbf{x} + \mathbf{c} - \mathbf{A}^T\boldsymbol{\lambda} = 0$$
$$\nabla_{\boldsymbol{\lambda}}\mathcal{L} = \mathbf{A}\mathbf{x} - \mathbf{b} = 0$$

This yields the following KKT system:

$$\begin{bmatrix} \mathbf{Q} & -\mathbf{A}^T \\ \mathbf{A} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} -\mathbf{c} \\ \mathbf{b} \end{bmatrix}$$

The KKT matrix is nonsingular because:

- The constraint matrix $\mathbf{A}$ is full rank (rows are linearly independent).
- The matrix $\mathbf{Q}$ is positive definite (symmetric with positive eigenvalues).

Numerical Solution and Interpretation

Plugging in the values:

$$\begin{bmatrix} 6 & 2 & 1 & -1 & 0 \\ 2 & 5 & 2 & 0 & -1 \\ 1 & 2 & 4 & -1 & -1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \lambda_1 \\ \lambda_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 3 \\ 0 \end{bmatrix}$$

Solving this system numerically (using Python, see `Q2.py`), we obtain:

$$x_1 \approx 1.462, \quad x_2 \approx -1.538, \quad x_3 \approx 1.538$$

The corresponding Lagrange multipliers are:

$$\lambda_1 \approx 6.231, \quad \lambda_2 \approx -2.692$$

The optimal value of the objective function is:

$$f(x_1, x_2, x_3) \approx 8.615$$

According to second-order optimality theory, the solution

$$\mathbf{x}^* \approx \begin{bmatrix} 1.462 & -1.538 & 1.538 \end{bmatrix}^T$$

is a **strict local minimum**, because the matrix $\mathbf{Q}$ is positive definite everywhere including the null-space of $\mathbf{A}$.

Question 3

Consider the constrained optimization problem:

$$\begin{aligned} \underset{x_1\in\mathbb{R},\ x_2\in\mathbb{R}}{\textbf{Minimize:}} \quad & f(x_1, x_2) = x_1(1 - x_2^2) \\ \textbf{Subject to:} \quad & h(x_1, x_2) = x_1^2 + x_2^2 - 1 = 0 \end{aligned}$$

First-order KKT Conditions

The Lagrangian function is:

$$\mathcal{L}(x_1, x_2, \lambda) = x_1(1 - x_2^2) - \lambda(x_1^2 + x_2^2 - 1)$$

Compute the gradients:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial x_1} &= 1 - x_2^2 - 2\lambda x_1 = 0 \\ \frac{\partial \mathcal{L}}{\partial x_2} &= -2x_1 x_2 - 2\lambda x_2 = 0 \\ \frac{\partial \mathcal{L}}{\partial \lambda} &= x_1^2 + x_2^2 - 1 = 0\end{aligned}$$

From the second equation:

$$-2x_2(x_1 + \lambda) = 0 \Rightarrow x_2 = 0 \quad \text{or} \quad x_1 = -\lambda$$

- Case 1: $x_2 = 0$

  From the 3rd condition: $x_1^2 = 1 \Rightarrow x_1 = \pm 1$

  From the 1st condition: $1 - 0 - 2\lambda x_1 = 0 \Rightarrow \lambda = \frac{1}{2x_1} = \pm\frac{1}{2}$

  Evaluate the objective:

  $x_1 = 1 \Rightarrow f = 1$

  $x_1 = -1 \Rightarrow f = -1$

- Case 2: $x_1 = -\lambda$

  From the 1st condition: $1 - x_2^2 - 2\lambda x_1 = 1 - x_2^2 + 2\lambda^2 = 0 \Rightarrow x_2^2 = 1 + 2\lambda^2$

  From the 3rd condition: $x_1^2 + x_2^2 = 1 \Rightarrow \lambda^2 + x_2^2 = 1$

  Substitute x_2^2 into that:

  $$\lambda^2 + (1 + 2\lambda^2) = 1 \Rightarrow 3\lambda^2 = 0 \Rightarrow \lambda = 0 \Rightarrow x_1 = 0 \Rightarrow x_2^2 = 1 \Rightarrow x_2 = \pm 1$$

  Evaluate the objective: $f = 0$

Summary of candidate points:

(x_1, x_2)	λ	$f(x_1, x_2)$
$(1, 0)$	$1/2$	1
$(-1, 0)$	$-1/2$	-1
$(0, \pm 1)$	0	0

Conclusion: Minimum is at $\boxed{(-1, 0)}$ with $f^* = -1$.

Modified Constraint: $(x_1^2 + x_2^2 - 1)^2 = 0$

This is an equivalent constraint in value, but it changes the structure of the KKT conditions because now the constraint is:

$$g(x_1, x_2) = (x_1^2 + x_2^2 - 1)^2 = 0$$

New Lagrangian:

$$\mathcal{L}(x_1, x_2, \lambda) = x_1(1 - x_2^2) - \lambda(x_1^2 + x_2^2 - 1)^2$$

Take gradients and set them to 0:

$$\frac{\partial \mathcal{L}}{\partial x_1} = 1 - x_2^2 - 4\lambda x_1(x_1^2 + x_2^2 - 1) = 0$$
$$\frac{\partial \mathcal{L}}{\partial x_2} = -2x_1x_2 - 4\lambda x_2(x_1^2 + x_2^2 - 1) = 0$$
$$\frac{\partial \mathcal{L}}{\partial \lambda} = (x_1^2 + x_2^2 - 1)^2 = 0$$

The last condition implies:

$$x_1^2 + x_2^2 = 1$$

Now plug this into the other derivatives:

$$\frac{\partial \mathcal{L}}{\partial x_1} = 1 - x_2^2 = 0 \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial x_2} = -2x_1x_2 = 0$$

So, the stationary conditions are:

$$1 - x_2^2 = 0 \Rightarrow x_2 = \pm 1, \quad x_1x_2 = 0 \Rightarrow x_1 = 0$$

Hence, the only KKT candidates are:

$$(x_1, x_2) = (0, \pm 1), \quad f = 0$$

But we already found a better solution at $(-1, 0)$, which satisfies the constraint $x_1^2 + x_2^2 - 1 = 0$, and hence also satisfies the modified constraint $(x_1^2 + x_2^2 - 1)^2 = 0$.
The modified KKT conditions only recovered part of the feasible set, but the true minimum is still:

$$\boxed{f^* = -1 \text{ at } (x_1, x_2) = (-1, 0)}$$

Why did the KKT conditions fail to recover the true minimum?
Recall from the definition of regularity that a point $\mathbf{x}^*$ is said to be *regular* if the gradients of the active constraints at that point are linearly independent.
In the original problem, the constraint was:

$$h(x) = x_1^2 + x_2^2 - 1 = 0$$

with gradient $\nabla h(x) = \begin{bmatrix} 2x_1 & 2x_2 \end{bmatrix}^T$, which is nonzero and linearly independent — so regularity holds at all feasible points (the origin is not a feasible point).
However, with the modified constraint, the gradient of the constraint becomes:

$$\nabla g(x) = 2(x_1^2 + x_2^2 - 1) \cdot \nabla(x_1^2 + x_2^2 - 1) = 4(x_1^2 + x_2^2 - 1) \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Since all feasible points satisfy $x_1^2 + x_2^2 - 1 = 0$, we have $\nabla g(x) = \mathbf{0}$ for all feasible $\mathbf{x}$. This means that the constraint gradient is zero; hence, the set of constraint gradients is linearly dependent. As a result, the regularity condition is violated. Therefore, the KKT conditions do not necessarily hold at optimal points. That's why the true minimizer $(-1, 0)$, although feasible and optimal, is not detected by the KKT conditions derived from the modified constraint.

Question 4

Let

$$J(\mathbf{x}) = \int_{\mathbb{R}^d} f(\mathbf{x}, \boldsymbol{\theta})\, p(\boldsymbol{\theta})\, d\boldsymbol{\theta}, \qquad \mathbf{x} \in \mathbb{R}^n,$$

be the Bayes risk to be minimised.

Computational challenges

a. **Expensive inner expectation.** Each design iterate $\mathbf{x}_k$ requires evaluating the high-dimensional integral—typically by Monte-Carlo (MC) quadrature—because f is a black box. When the dimension d of $\boldsymbol{\theta}$ is large, the number of samples N required for an accurate estimate of J grows rapidly (curse of dimensionality).

b. **Noisy/biased objective values.** MC evaluation yields $\widehat{J}(\mathbf{x}_k)$ with sampling error $\mathcal{O}(N^{-1/2})$. This noise can mislead line–search or trust-region tests and deteriorate convergence rates.

c. **Unavailable or costly gradients.** Even if f is smooth in $\mathbf{x}$, differentiating *through* the integral requires either computing $\nabla_{\mathbf{x}} J = \int \nabla_{\mathbf{x}} f(\mathbf{x}, \boldsymbol{\theta})\, p(\boldsymbol{\theta})\, d\boldsymbol{\theta}$ or finite-difference/complex-step approximations. Both cases need many additional black-box calls ($\mathcal{O}(nN)$ per iteration).

d. **Non-convex, multi-modal landscape.** Robust design criteria often yield flat or multi-modal $J(\mathbf{x})$, so purely local, deterministic methods risk premature convergence.

e. **Parallelism vs. synchronisation.** Although the integral is parallel, synchronous deterministic optimizers (exact line search SQP) may idle while awaiting the slowest sample.

Suitable methods from AER1415

1. Stochastic non-gradient search (SA / PSO / EA).

- *Rationale*: Simulated Annealing, Particle Swarm Optimization, and Evolutionary Algorithms do not need derivatives and tolerate noisy $\widehat{J}$ evaluations—exactly the situation produced by MC estimates of J.
- *Implementation*: Each candidate $\mathbf{x}$ is scored with the same sample set (common random numbers) to reduce variance, while population–based search naturally amortises the N samples across processors.
- *Trade-off*: Global exploration mitigates multi-modality, but convergence is only probabilistic and the number of function calls can still be large. Hence they are well–suited when n is modest and f is truly a black-box.

2. Sample-average & quasi-Newton (BFGS / L–BFGS).

- *Rationale*: If $\nabla_{\mathbf{x}} f$ is available (via automatic differentiation or finite-difference/complex-step approximations), the sample-average approximation $\widehat{J}(\mathbf{x}) = \frac{1}{N}\sum_{i=1}^{N} f(\mathbf{x}, \boldsymbol{\theta}^{(i)})$ is smooth; quasi-Newton methods (BFGS, L–BFGS) then exploit super-linear convergence.
- *Variance control*: Increase N or employ a trust-region on N as $\|\nabla \widehat{J}\|$ shrinks, following inexact Newton ideas.

- *When effective*: Recommended when gradient information is cheap relative to function evaluation and n is moderate–to–large (where L-BFGS shines).

Recommended Strategy: Run a global stochastic search (e.g. PSO) with a coarse MC budget to locate promising basins; then switch to an L–BFGS optimizer with gradually increasing N to obtain a high-accuracy robust design. This hybrid strategy leverages the strengths of both algorithm classes presented in AER1415 while directly addressing the challenges listed above.

Question 5

SGD Optimization Algorithm Implementation

The SGD optimization algorithm was implemented in the Python script `Q5.py` as follows:
Let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ be the training set, $d = 5$ the number of inputs, and $m = 64$ the width of the hidden layer. All network parameters are collected in a single vector

$$\theta = \big[\underbrace{\mathrm{vec}(W_1)}_{d\times m},\ \underbrace{b_1}_{m},\ \underbrace{W_2}_{m},\ \underbrace{b_2}_{1}\big]^{\top} \in \mathbb{R}^n, \qquad n = dm + 2m + 1.$$

Loss and gradients. For a mini-batch $\mathcal{B} \subset \mathcal{D}$ of size B we evaluate the mean–squared-error loss

$$\mathcal{L}(\theta; \mathcal{B}) = \frac{1}{B} \sum_{(x,y)\in\mathcal{B}} \big(y - \hat{y}(x;\theta)\big)^2,$$

together with its gradient $\nabla_\theta \mathcal{L}(\theta; \mathcal{B})$ via the provided routine `loss.py`.

Parameter update. With a fixed learning rate $\eta > 0$ the parameters are updated as

$$\boxed{\theta_{k+1} \;=\; \theta_k \;-\; \eta\, \nabla_\theta \mathcal{L}\big(\theta_k; \mathcal{B}_k\big)} \qquad \text{for } k = 0, \ldots, K-1,$$

where $\mathcal{B}_k$ is the k-th mini-batch drawn during the current epoch.

Epoch structure and shuffling.

1. **Shuffle:** At the start of every epoch we randomly permute the training indices to reduce correlation between successive batches.
2. **Mini-batch loop:** The permuted data are traversed in contiguous blocks of size B. For each block, we call `loss(theta, X_batch, y_batch, d, m)` to obtain the loss value and its gradient, then apply the update above.
3. **Book-keeping:** After completing all mini-batches we compute the *full-batch* training loss for monitoring and store it in a learning-curve array.

Initialization and normalization.

- Inputs are standardized: $x \leftarrow (x - \mu)/\sigma$ with μ, σ computed from the *training* set only.
- Weights are initialized randomly, matching the conjugate gradient method script.

Stopping rule. The optimizer runs for exactly 1000 epochs without additional early-stopping criteria.

Mini-batch SGD with constant learning rate

input: training set D, batch size B, learning rate η, epochs K
initialise θ_0 (see above)
for $k = 0, \ldots, K-1$ **do** ▷ epoch loop
    shuffle D and split into mini-batches
    for all mini-batches $\mathcal{B}$ **do**
        compute $(\mathcal{L}, g) = \texttt{loss}(\theta_k, \mathcal{B})$
        $\theta_k \leftarrow \theta_k - \eta\, g$ ▷ SGD step
    end for
    store full-batch loss for diagnostics
end for
return final parameters θ_K

SGD Optimization Algorithm Results

Experimental set-up. For each $(B, \eta) \in \{16, 32, 64\} \times \{10^{-4}, 10^{-3}, 10^{-2}\}$ we trained the 449-parameter network for 1000 epochs with no early stopping. Every hyper-parameter pair was repeated with 10 independent seeds; the mean and sample standard-deviation of the mean–square error (MSE) are reported in Table 1. Loss and gradient-norm trajectories averaged over seeds are shown in Figs. 1–2.

Influence of learning rate. A very small step size ($\eta = 10^{-4}$) converges smoothly but slowly, plateauing above ≈ 10 MSE. Increasing to $\eta = 10^{-3}$ accelerates convergence and achieves the best overall trade-off between speed and stability. A further increase to $\eta = 10^{-2}$ speeds up the initial decrease but introduces large oscillations (Fig. 2, right), yielding larger seed-to-seed variance despite the lowest mean MSE.

Influence of batch size. Across all learning rates, the smallest mini-batch ($B = 16$) yields the lowest training and test MSEs, e.g. $3.06 \pm 0.18/4.49 \pm 0.24$ at $\eta = 10^{-3}$. The additional gradient noise with small B acts as an implicit regularizer, reducing over-fitting.

Best configuration. Considering convergence speed, gradient stability, and generalization, the combination $\eta = 10^{-3}$, $B = 16$ is preferred.

SGD Comparison to CG

Overview. To benchmark the performance of stochastic gradient descent (SGD), we compared it against the provided conjugate gradient (CG) baseline. Both methods were run for 10 random seeds, and their final training and test mean–square errors (MSEs) are shown in Table 1. Convergence plots for the loss and gradient norm are shown in Figs. 1 and 2, with CG overlaid for comparison.

Accuracy comparison. The CG baseline achieves a final test MSE of 5.05 ± 0.95, which is competitive with the best-performing SGD configurations. However, the smallest mini-batch SGD setting ($\eta = 10^{-2}, B = 16$) attains an even lower test MSE of 3.85 ± 0.42. This indicates that when tuned properly, SGD not only matches but can surpass CG in terms of generalization.

Table 1: Mean–square error (*MSE*) and sample standard deviation σ over 10 runs, reported for each mini-batch size B (rows) and learning rate η (column groups). The bottom row reports the baseline conjugate gradient (CG) results.

	$\eta = 10^{-4}$		$\eta = 10^{-3}$		$\eta = 10^{-2}$	
B	Train	Test	Train	Test	Train	Test
16	8.68 ± 0.40	10.55 ± 0.45	3.06 ± 0.18	4.49 ± 0.24	1.90 ± 0.20	3.85 ± 0.42
32	11.71 ± 0.84	14.22 ± 0.99	4.11 ± 0.19	5.37 ± 0.23	2.17 ± 0.22	3.96 ± 0.27
64	14.62 ± 0.58	17.78 ± 0.72	5.65 ± 0.18	6.96 ± 0.20	2.41 ± 0.28	4.18 ± 0.31
CG	2.70 ± 0.87	5.05 ± 0.95	2.70 ± 0.87	5.05 ± 0.95	2.70 ± 0.87	5.05 ± 0.95

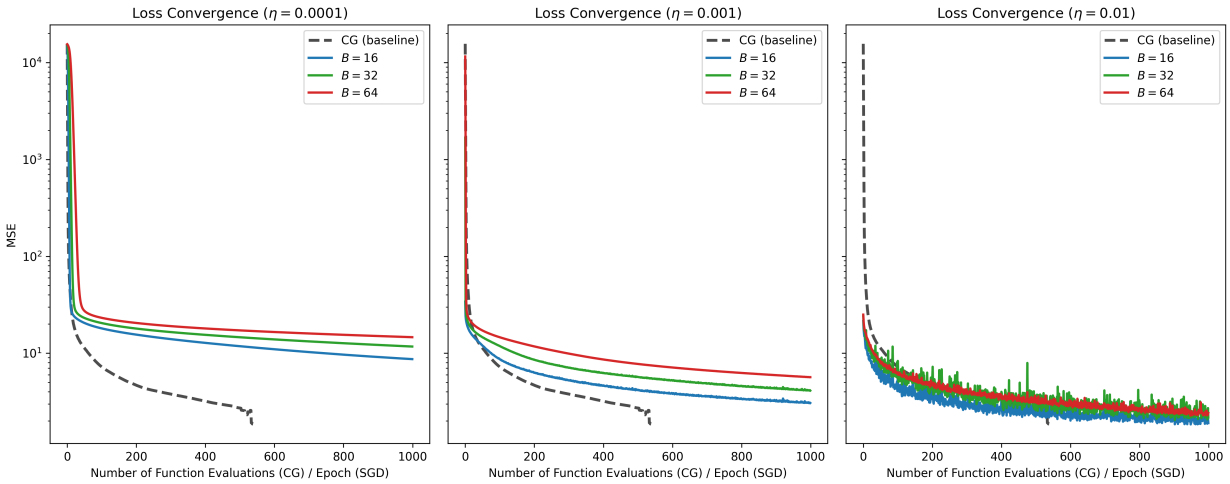


Figure 1: Training loss over epochs for different batch sizes, grouped by learning rate.

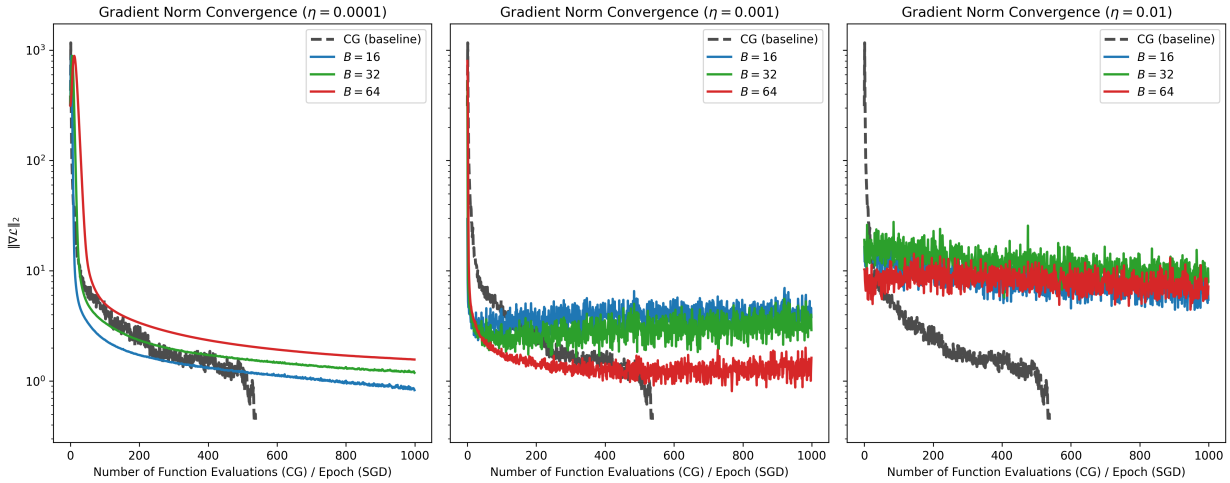


Figure 2: Training norm of gradient over epochs for different batch sizes, grouped by learning rate.

Convergence speed. From Fig. 1, CG initially decreases the loss more steeply than SGD and reaches a low gradient norm quickly (see Fig. 2). This is expected since CG is a second-order

optimization method, making better use of curvature information. However, its progress plateaus early, while SGD continues to improve — particularly in the lower learning-rate settings.

Gradient stability. The gradient norm plots reveal that CG quickly drives the gradient to near-zero in fewer iterations, while SGD shows persistent gradient oscillations — especially at higher learning rates ($\eta = 10^{-2}$). However, these oscillations do not necessarily impair generalization, as shown by the test MSEs.

Practical trade-offs. While CG converges in fewer iterations (typically fewer than 550 function evaluations), it is computationally more intensive per step and less scalable to large datasets. In contrast, SGD trains efficiently in fixed epoch budgets and is much more adaptable to mini-batch processing and parallelization.

Conclusion. The CG method provides a strong baseline and demonstrates fast initial convergence. However, when tuned appropriately, SGD achieves better generalization and scales more flexibly to larger problems. For this task, the best-performing SGD configuration outperforms CG both in final test error and in training efficiency per compute cost.

Question 6

Hybrid PSO–SLSQP Optimization Algorithm

Overview. We propose a hybrid constrained optimization algorithm that integrates Particle Swarm Optimization (PSO) with the local solver `SLSQP` from `scipy.optimize`. Instead of using PSO merely to initialize a local optimizer, our algorithm interleaves global exploration by PSO with periodic local refinements using SLSQP to accelerate convergence and enhance precision near optima.

PSO Formulation. The core of our algorithm is based on the standard PSO formulation, in which a swarm of n_p particles iteratively updates their velocities and positions according to:

$$\mathbf{v}_i(t{+}1) = w\mathbf{v}_i(t) + c_1\mathbf{r}_1 \odot (\mathbf{p}_{i,\text{best}} - \mathbf{x}_i(t)) + c_2\mathbf{r}_2 \odot (\mathbf{g}_{\text{best}} - \mathbf{x}_i(t)),$$
$$\mathbf{x}_i(t{+}1) = \mathbf{x}_i(t) + \mathbf{v}_i(t{+}1),$$

where:

- $\mathbf{x}_i(t)$ and $\mathbf{v}_i(t)$ denote the position and velocity of particle i at iteration t,
- w, c_1, c_2 are the inertia and acceleration coefficients,
- $\mathbf{r}_1$, $\mathbf{r}_2$ are vectors of uniform random variables in $[0, 1]$,
- $\mathbf{p}_{i,\text{best}}$ and $\mathbf{g}_{\text{best}}$ are the personal and global best solutions.

Constraint Handling. Constraints are handled using a *static penalty method*. The penalized objective function is defined as:

$$f_\rho(\mathbf{x}) = f(\mathbf{x}) + \rho \cdot \sum_j \max(0,\, g_j(\mathbf{x}))^2 + \rho \cdot \sum_k h_k(\mathbf{x})^2,$$

where $g_j(\mathbf{x})$ are inequality constraints and $h_k(\mathbf{x})$ are equality constraints. A fixed penalty weight $\rho = 7535$ was used throughout, as determined in Assignment 1.

Tuned Parameters. Based on prior tuning on the Bump test function:

- Penalty method: Static
- n_p: 59 particles
- w: 0.406
- c_1: 1.527
- c_2: 2.135
- ρ: 7535

Hybridization Strategy. To combine global and local search, we periodically refine the best particles found by PSO using the SLSQP solver. This allows the swarm to explore widely while ensuring that high-potential regions are locally optimized using gradient information.

User-defined hybrid parameters.

- T: Total number of PSO iterations
- k: Frequency (in PSO iterations) of local refinements
- n_{refine}: Number of elite particles to refine using SLSQP

Pseudocode.

Initialize particle positions $\mathbf{x}_i(0)$ and velocities $\mathbf{v}_i(0)$
Evaluate penalized objective $f_\rho(\mathbf{x}_i)$ for all particles
Store $\mathbf{p}_{i,\text{best}}$, $\mathbf{g}_{\text{best}}$
for $t = 1$ to T **do**
    for each particle i **do**
        Update $\mathbf{v}_i$ and $\mathbf{x}_i$ using PSO update rules
        Evaluate $f_\rho(\mathbf{x}_i)$ and update $\mathbf{p}_{i,\text{best}}$
    end for
    Update global best $\mathbf{g}_{\text{best}}$
    if $t \mod k = 0$ **then**
        Select top n_{refine} particles
        for each selected particle **do**
            Apply `scipy.optimize.minimize(method='SLSQP')` from $\mathbf{x}_i$
            Update $\mathbf{g}_{\text{best}}$ if SLSQP improves global best
        end for
    end if
end for
return best solution found $\mathbf{g}_{\text{best}}$

Benefits. This hybrid scheme achieves a balance between global exploration and local exploitation. The SLSQP solver improves convergence accuracy, while the PSO swarm maintains robustness to local minima. The synergy improves solution quality, especially for constrained optimization problems with complex feasible regions.

Hybrid PSO–SLSQP Applied to the Bump Function (P3)

Experimental setup. The proposed hybrid PSO–SLSQP algorithm was applied to the 20-dimensional constrained Bump test function (P3), using the static penalty method and hyper-parameters tuned in Assignment 1. The hybrid parameters used were $k = 100$ (SLSQP refinement every 100 iterations) and $n_{\text{refine}} = 3$ (top 3 particles refined). Both the standard PSO and the hybrid PSO–SLSQP were run for 10 independent seeds with a maximum of 1000 iterations.

Performance comparison. Table 2 summarizes the performance statistics across 10 runs for both methods. The hybrid approach shows a slight improvement in mean objective value and reduced variance compared to the standard PSO.

Table 2: Comparison of performance statistics for the standard PSO and hybrid PSO–SLSQP on the Bump test function (P3).

Method	**Mean**	**Std**	**Min**	**Max**
Standard PSO	-0.5175	0.1574	-0.7461	-0.2736
Hybrid PSO–SLSQP	-0.5286	0.1481	-0.7463	-0.2837

Convergence analysis. The convergence behavior of both algorithms is illustrated in Fig. 3. The hybrid approach converges faster within the first 200 iterations and achieves slightly better mean performance overall. The use of periodic SLSQP refinements improves the stability and quality of solutions.

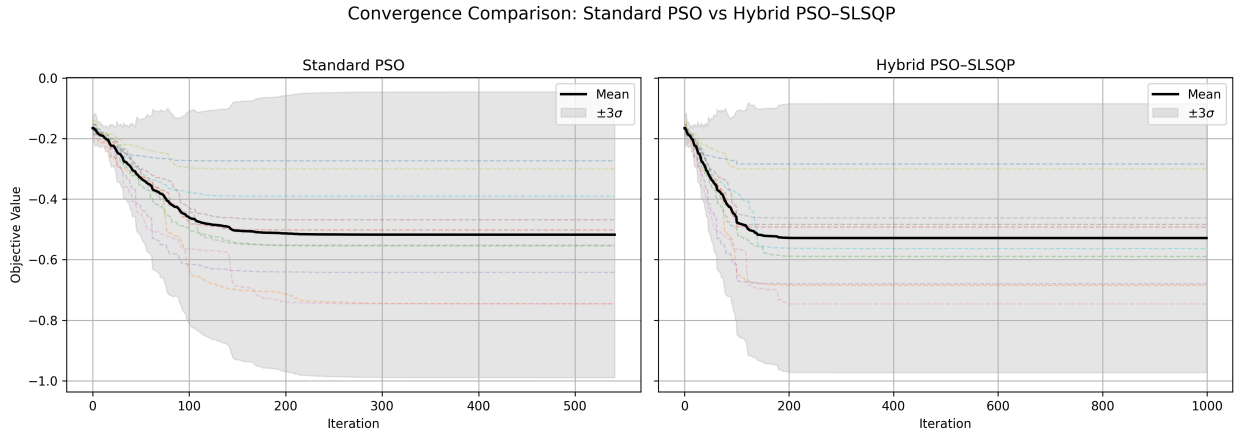


Figure 3: Convergence comparison of standard PSO and hybrid PSO–SLSQP on the Bump test function (P3). The shaded area represents $\pm 3\sigma$ across 10 runs.

Conclusion. While both algorithms performed comparably, the hybrid PSO–SLSQP showed modest improvements in convergence rate and robustness, achieving a better mean objective value and tighter variability across independent runs. This confirms that hybridization with a local solver like SLSQP can enhance PSO performance on constrained optimization problems.

Question 7

Problem statement

Let the *optimal* design that has already been obtained be the vector

$$\mathbf{x}^\star \;=\; \arg\min_{\mathbf{x}\in\mathcal{X}} \; f(\mathbf{x}), \qquad \mathcal{X} := \big\{\mathbf{x}\in\mathbb{R}^n : g_j(\mathbf{x}) \le 0,\; j = 1,\dots,m\big\},$$

where

- $f : \mathbb{R}^n \to \mathbb{R}$ is the performance (or cost) metric to be *minimised*,
- $g_j : \mathbb{R}^n \to \mathbb{R},\; j = 1,\dots,m$ are inequality constraints that must always be satisfied.

In production, each design variable is allowed to deviate from its nominal optimal value $\mathbf{x}^\star$ by at most a (non–negative) amount $\boldsymbol{\delta} = \big[\delta_1,\dots,\delta_n\big]^\mathsf{T}$, giving the *tolerance box*

$$\mathcal{T}(\boldsymbol{\delta}) \;=\; \big\{\mathbf{x} : |x_i - x_i^\star| \le \delta_i,\; i = 1,\dots,n\big\}.$$

The aim is to find the **largest** vector $\boldsymbol{\delta}$ such that

1. **Worst-case performance bound.** For every perturbed design inside $\mathcal{T}(\boldsymbol{\delta})$ the performance does not deteriorate by more than 5%, i.e.

$$\sup_{\mathbf{x}\in\mathcal{T}(\boldsymbol{\delta})} f(\mathbf{x}) \;\le\; (1+\eta)\, f(\mathbf{x}^\star), \qquad \eta = 0.05;$$

2. **Feasibility bound.** For every perturbed design inside $\mathcal{T}(\boldsymbol{\delta})$ *all* constraints remain satisfied:

$$\sup_{\mathbf{x}\in\mathcal{T}(\boldsymbol{\delta})} g_j(\mathbf{x}) \;\le\; 0, \qquad j = 1,\dots,m.$$

Optimisation formulation

A convenient scalar measure of the overall tolerance size is the ℓ_∞–radius[1] $r \;=\; \max_{i=1,\dots,n}\delta_i$. We seek the largest r (and the associated $\boldsymbol{\delta}$) that satisfies the two worst–case requirements above. The resulting *robust* optimisation problem can be written compactly as

$$\boxed{\begin{aligned}
&\max_{r,\,\boldsymbol{\delta}} \; r \\
&\text{s. t. } 0 \le \delta_i \le r, \qquad i = 1,\dots,n, \\
&\qquad \sup_{\substack{|x_i - x_i^\star|\le\delta_i \\ i=1,\dots,n}} f(\mathbf{x}) \;\le\; (1+\eta)\, f(\mathbf{x}^\star), \\
&\qquad \sup_{\substack{|x_i - x_i^\star|\le\delta_i \\ i=1,\dots,n}} g_j(\mathbf{x}) \;\le\; 0, \qquad j = 1,\dots,m.
\end{aligned}} \tag{$\mathrm{P_{tol}}$}$$

Decision variables.

- $r \in \mathbb{R}_{\ge 0}$ – the half–width of the largest identical bilateral tolerance that we wish to maximise;
- $\boldsymbol{\delta} \in \mathbb{R}^n_{\ge 0}$ – individual half–widths in each coordinate direction (bounded above by r).

[1] Any other norm (e.g. weighted ℓ_1 or ℓ_2) could be used with trivial modifications.

Remarks and practical variants

- **No constraints case.** If $m = 0$, problem ($\mathrm{P}_{\mathrm{tol}}$) reduces to a single worst–case performance constraint.
- **Analytical reformulation.** When f and g_j are differentiable one may replace the semi–infinite constraints by first–order Taylor upper bounds, e.g. $f(\mathbf{x}) \leq f(\mathbf{x}^\star) + \left\|\nabla f(\mathbf{x}^\star)\right\|_1 r$, leading to a conservative but linear (or convex) programme in $(r, \boldsymbol{\delta})$.
- **Scenario/Monte-Carlo approach.** In black–box settings the sup operators can be approximated by a finite set of samples $\{\mathbf{x}^{(s)}\}_{s=1}^{S} \subset \mathcal{T}(\boldsymbol{\delta})$, yielding a tractable nonlinear programme.
- **Simultaneous co-design.** One may solve for the nominal design *and* its tolerances in a single optimisation by adding $\mathbf{x}^\star$ as another decision variable and appending the original cost $f(\mathbf{x}^\star)$ to the objective.

The boxed formulation ($\mathrm{P}_{\mathrm{tol}}$) captures the engineering trade–off between production feasibility (large r) and guaranteed functional performance.

Bonus Question

Objective. To further improve the convergence behaviour of the mini-batch SGD algorithm from Question 5, we implemented a variant with classical momentum. This aims to accelerate convergence by incorporating a velocity term that smooths out oscillations in the gradient updates.

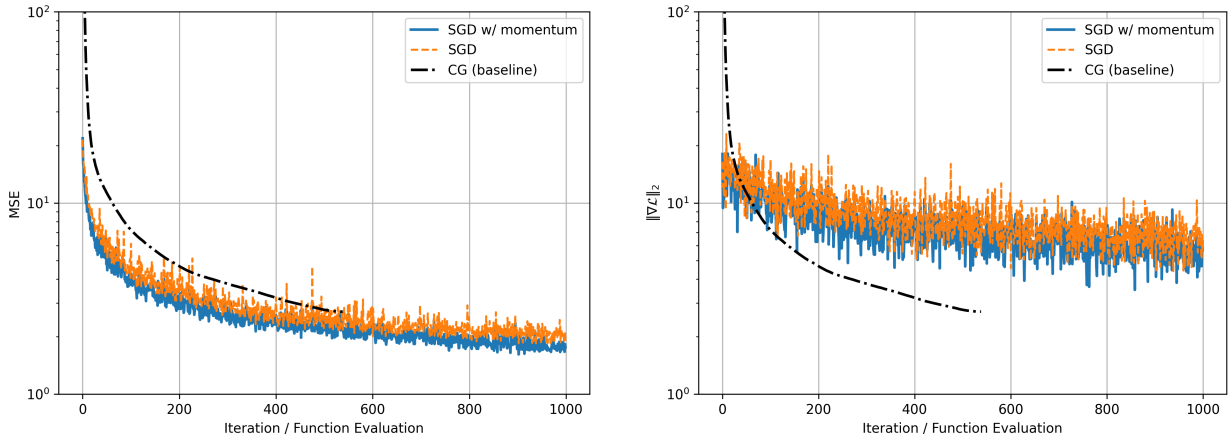
Experimental set-up. Both the standard SGD and the momentum-based SGD used a batch size of $B = 16$ and learning rate $\eta = 10^{-2}$. The momentum coefficient was fixed at $\beta = 0.5$ for all experiments. Each method was run over 10 independent seeds for 1000 epochs. The baseline Conjugate Gradient (CG) method was included for comparison.

Results. Table 3 summarizes the train/test mean square errors (MSEs) averaged over the 10 seeds for each method. MOM achieved the lowest test MSE and exhibited reduced variability compared to plain SGD. The CG baseline converged rapidly but to a higher test error, indicating possible over-fitting.

Table 3: Performance statistics of SGD with momentum (MOM), standard SGD, and the CG baseline. Results are averaged over 10 seeds.

Method	Train MSE	$\pm\sigma$	Test MSE	$\pm\sigma$
SGD w/ momentum	1.7594	0.3078	3.8362	0.3255
SGD	1.9004	0.1981	3.8508	0.4160
CG (baseline)	2.7020	0.8739	5.0523	0.9502

Discussion. The addition of momentum slightly improved convergence speed and stability, as shown in Figure 4. The loss curves demonstrate that momentum helps dampen oscillations, leading to smoother and more consistent descent. The gradient norms also decayed faster with momentum-based SGD, indicating more efficient optimization.
The CG method exhibited the fastest initial convergence due to second-order information, but its final test performance lagged behind the SGD variants. This suggests that first-order methods with mini-batch stochasticity may generalize better in this setting.



(a) Loss convergence.

(b) Gradient norm convergence.

Figure 4: Comparison of SGD with momentum, standard SGD, and CG.